

Raspberry Pi GPIO Access Using C

ONKAR MATE

It has almost been 40 years or more and still C language is ruling the world. Also, it has been the first choice of embedded engineers. It is simple to learn and a comfort zone for many who have already learnt it and do not want to switch to a new language. So let us develop applications on Raspberry Pi using this language.

For electronics enthusiasts, the main area of interest is the general-purpose input/output (GPIO) port of Raspberry Pi. It is a generic pin on a chip and its status (that is, input/output (I/O) or logic levels) can be controlled/programmed through software. GPIO pins have great importance in the embedded world as you can interface lower-level peripherals and control them by manipulating at bit level.



Being a hardware resource, GPIOs require assembly languages (which are specific to that controller) to control them. If one wants to use a higher level programming language to control them then additional header/library files are needed, which will map functionality from higher level languages to Assembly language.

Raspberry Pi supports a number of programming languages such as C, C++, Ruby, Perl, Python, Java and PHP but, to access GPIO, all of them require additional libraries.

Due to availability of cross compilers for C, it is possible to program microcontrollers with different architectures without learning individual Assembly languages. Accessing I/Os, bit manipulations and direct hardware control is possible in C with simplicity like higher level languages at the same time.

Raspberry Pi is not an exception in the world of embedded systems. Thanks to Gordon Henderson who developed C library (called WiringPi)

for Raspberry Pi to facilitate its GPIO access using C language.

About GPIOs of Raspberry Pi

The Raspberry Pi has a 26-pin GPIO connector. There are eight GPIO pins which can be programmed as either digital outputs or inputs. One of these pins can be designated for PWM output too.

Additionally, there is a 2-wire I²C interface and a 4-wire SPI interface. The I²C and SPI interfaces can also be used as GPIO pins when they are not being used in their bus modes, and the UART pins can also be used if you reboot with the serial console disabled. Fig. 1 shows the Raspberry Pi header.

Note here that pins in white colour are unused pins. These pins are reserved for future use and should not be connected to at this point in time

Getting started with WiringPi

WiringPi is an Arduino wiring-like library (part of the Arduino software package called Wiring that handles input and output for the Arduino) written in C and released under the GNU LGPLv3 licence, which is usable from C and C++ and many other languages

with suitable wrappers.

To develop C applications with WiringPi a compiler is needed. Usually, GNU compiler collection (GCC) is already built in Linux distribution that runs on Raspberry Pi.

WiringPi library needs to be installed on Raspberry

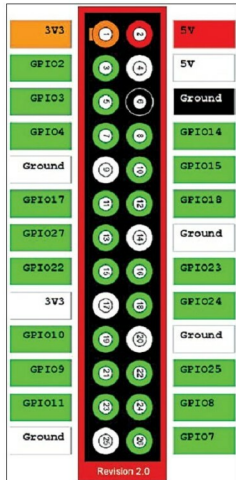


Fig. 1: GPIOs of Raspberry Pi header (main header)

TABLE I
Different Naming Schemes for Raspberry Pi Pins

WiringPi pin	BCM GPIO	Name	Header	Name	BCM GPIO	WiringPi pin
—	—	3.3V	1 2	5V	—	—
8	2	SDA0	3 4	DNC	—	—
9	3	SCL0	5 6	0V	—	—
7	4	GPIO7	7 8	TxD	14	15
—	—	DNC	9 10	RxD	15	16
0	17	GPIO0	11 12	GPIO1	18	1
2	27	GPIO2	13 14	DNC	—	—
3	22	GPIO3	15 16	GPIO4	23	4
—	—	DNC	17 18	GPIO5	24	5
12	10	MOSI	19 20	DNC	—	—
13	9	MISO	21 22	GPIO6	25	6
14	11	SCLK	23 24	CE0	8	10
—	—	DNC	25 26	CE1	7	11